ACTIVIDAD NO. 2: PATRONES DE COMPORTAMIENTO
REUTILIZACIÓN DE SOFTWARE

Sistema de noticias con el patrón Observer

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
DEIVIS EDUARD RAMÍREZ MARTÍNEZ

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
8 DE NOVIEMBRE
2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN 3
2. EJERCICIO 1: VENTAJAS E INCONVENIENTES DEL PATRÓN 3
2.1 Implementación sin patrón y sus problemas 3
2.2 Solución con Observer 5
2.3 Ventajas 6
2.4 Inconvenientes 7
2.5 Alternativas consideradas 7
2.6 Reutilización de software en la solución 8
3. EJERCICIO 2: DISEÑO E IMPLEMENTACIÓN 8
3.1 Participantes del patrón 8
3.2 Diagrama UML del sistema 9
3.3 Implementación en Java 10
3.4 Decisiones de diseño 14
3.5 Resultados de la ejecución 14
4. DESARROLLO ADICIONAL: MÁS ALLÁ DEL ENUNCIADO 15
4.1 Evolución del sistema: nuevo canal Email 15
4.2 Pruebas automatizadas, incluida una prueba de arquitectura 17
4.3 Alternativa implementada: PropertyChangeSupport 19
4.4 Guía de decisión y recomendación 20
5. CONCLUSIONES 20
6. BIBLIOGRAFÍA 21
7. ANEXO: ENTREGA DEL CÓDIGO Y EJECUCIÓN 22
8. AGRADECIMIENTO 22

FICHA DEL DOCUMENTO

Proyecto	Sistema de noticias (patrón Observer)
Documento	Actividad No. 2: Patrones de comportamiento
Asignatura	Reutilización de Software
Versión	1.0 (noviembre de 2026)
Fecha	8 de noviembre de 2026
Autor	Alejandro De Mendoza
Docente	Deivis Eduard Ramírez Martínez
Repositorio	https://github.com/AlejoTechEngineer/observer-sistema-noticias
Lenguaje	Java (JDK 16 o superior por el uso de records; probado con JDK 21)

1. INTRODUCCIÓN

La presente actividad tiene como propósito aplicar los patrones de diseño de comportamiento estudiados en los Temas 8, 9 y 10, mediante el análisis, diseño e implementación de un sistema de noticias. En el caso de estudio, varios reporteros envían noticias y es necesario notificar de manera automática a suscriptores de distintos tipos: un canal de televisión, una página web y notificaciones a través de una aplicación móvil. Cada reportero puede dirigirse a un conjunto diferente de suscriptores, lo que plantea un desafío de diseño relacionado con el acoplamiento entre quien produce la información y quienes la consumen, y con la capacidad de incorporar nuevos canales sin modificar el código ya existente.

Para abordar este escenario se estudia el patrón de diseño Observador (*Observer*), evaluando su pertinencia como mecanismo para definir una dependencia de uno a muchos entre objetos, de modo que un cambio de estado en el sujeto se notifique automáticamente a todos sus observadores. Asimismo, se contrasta esta solución con una implementación que no utiliza patrones de diseño, con el propósito de identificar sus diferencias en términos de acoplamiento, mantenibilidad, extensibilidad y costo de evolución, y se justifica por qué se descartan los demás patrones de comportamiento estudiados.

El desarrollo se organiza de acuerdo con los dos ejercicios establecidos en el enunciado. El Ejercicio 1 presenta el análisis comparativo de las ventajas, los inconvenientes y las alternativas de aplicación del patrón Observer frente a la solución básica. El Ejercicio 2 desarrolla el diseño de la solución mediante diagramas UML (clases y secuencia) y su implementación en Java, con 4 reporteros y 3 tipos de salida (Web, TV Channel y notificación a la app móvil), además de la simulación con entrada y salida por pantalla. Adicionalmente, el capítulo 4 amplía voluntariamente el alcance de la actividad mediante la evolución del sistema con un cuarto canal (Email) medida contra la versión sin patrón, una suite de 12 pruebas automatizadas que incluye una prueba de arquitectura y una alternativa implementada con java.beans.PropertyChangeSupport. Todas las cifras de este documento provienen de ejecuciones y mediciones realizadas sobre el código fuente de cada etapa.

Finalmente, el código fuente completo, organizado por etapas y acompañado de las instrucciones de compilación y ejecución, se encuentra disponible en el siguiente repositorio de GitHub: https://github.com/AlejoTechEngineer/observer-sistema-noticias (véase también el capítulo 7). La implementación se realiza en Java; sin embargo, los principios de diseño, las relaciones estructurales representadas mediante UML y los fundamentos del patrón Observer son aplicables a otros lenguajes de programación orientados a objetos.

2. EJERCICIO 1: VENTAJAS E INCONVENIENTES DEL PATRÓN

En este ejercicio se analiza la aplicación del patrón de diseño Observer en el sistema de noticias, identificando sus principales ventajas e inconvenientes frente a la solución básica del enunciado. Se examina su impacto en la organización del código, el acoplamiento, la mantenibilidad y la extensibilidad, con el propósito de determinar en qué medida este patrón aporta beneficios al diseño y a la evolución de la solución propuesta.

2.1 Implementación sin patrón y sus problemas

La solución básica (Figura 1) conecta cada reportero directamente con las salidas que le corresponden: cada clase de reportero crea las clases concretas Web, TVChannel y AppMovil y las invoca una por una cada vez que publica

una noticia. Esta es la versión implementada en sinpatron/ para el mismo escenario del enunciado; a modo de ejemplo se muestran los reporteros 1 y 3:

```
package com.unir.noticias.sinpatron;

/** Reportero 1: conoce y llama directamente a las salidas concretas (acoplamiento fuerte). */
public class Reportero1 {
    private final Web web = new Web();
    private final TVChannel tv = new TVChannel();
    private final AppMovil app = new AppMovil();

    public void publicarNoticia(String titular) {
        System.out.println("Esperando noticia: Reportero 1 - " + titular);
        web.publicar(titular);
        tv.emitir(titular);
        app.notificar(titular);
        System.out.println();
    }
}
```

```
package com.unir.noticias.sinpatron;

/** Reportero 3: conoce y llama directamente a las salidas concretas (acoplamiento fuerte). */
public class Reportero3 {
    private final Web web = new Web();

    public void publicarNoticia(String titular) {
        System.out.println("Esperando noticia: Reportero 3 - " + titular);
        web.publicar(titular);
        System.out.println();
    }
}
```

La versión sin patrón funciona y produce exactamente la misma salida que la solución con Observer (se comprobó con diff; véase la sección 4.3), pero presenta problemas estructurales que se agravan con la evolución del sistema:

- **Acoplamiento fuerte:** cada reportero conoce las clases concretas de las salidas. En este escenario hay 8 dependencias directas (reportero → salida concreta); con 4 reporteros y 3 salidas el máximo posible sería 12, y crece con el producto de ambas cantidades.
- **Violación del principio Abierto/Cerrado:** añadir una salida nueva obliga a *reabrir y modificar* las clases de los reporteros que deben usarla. Cada modificación arriesga regresiones en código que ya funcionaba (Tabla 1).
- **Duplicación de lógica:** la secuencia «avisar a Web, luego a TV, luego a App» se repite, con pequeñas variantes, en cuatro clases diferentes.
- **Suscripciones rígidas:** el conjunto de salidas de cada reportero queda fijado en tiempo de compilación; no es posible activar ni desactivar una salida durante la ejecución.
- **Baja reutilización y pruebas difíciles:** no se puede reutilizar un reportero sin arrastrar sus salidas, ni sustituirlas por dobles de prueba, porque se instancian dentro de la propia clase.

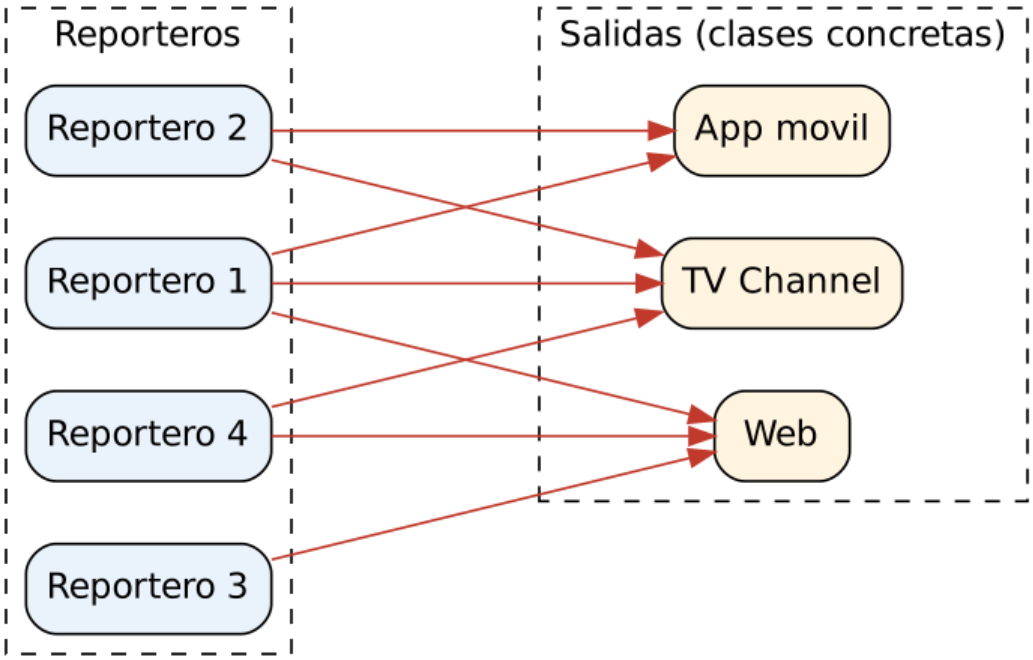


Figura 1. Solución básica sin patrón: cada reportero depende directamente de las salidas concretas que utiliza (8 dependencias directas).

Con el propósito de evidenciar el impacto de incorporar un nuevo canal, la Tabla 1 compara las dos versiones tras añadir una cuarta salida (Email) para los reporteros 1 y 4. Las cifras se midieron sobre el código fuente de cada versión (líneas no vacías, sin comentarios ni *imports*); la evolución se describe en la sección 4.1.

Tabla 1. Solución sin patrón frente a Observer: tamaño y efecto de añadir el canal Email (medido sobre sinpatron/ y etapa1_base/, etapa2_extension/).

Métrica	Sin patrón	Con Observer
Clases de reportero	4 (Reportero1 a Reportero4)	1 (Reportero, instanciada 4 veces)
Dependencias directas reportero → salida concreta	8	0
Líneas de código del núcleo (sin Main)	49	79
Archivos de reportero modificados al añadir Email	2 (Reportero1, Reportero4)	0
Archivos existentes modificados en total	2	1 (Main, punto de composición)
Líneas añadidas en archivos existentes	4	3
Archivos nuevos	1 (Email)	1 (CanalEmail)
Salida por pantalla	idéntica	idéntica

La tabla muestra un resultado que conviene señalar con honestidad: **a este tamaño el patrón no reduce el código; lo aumenta** (79 frente a 49 líneas en el núcleo). Su valor no está en escribir menos, sino en dónde recae el cambio: con Observer ninguna clase de reportero se modifica al añadir un canal, y el trabajo de «cablear» las suscripciones se concentra en un único punto de composición (Main) en lugar de repartirse por las clases de negocio.

2.2 Solución con Observer

Según el Tema 10, el patrón Observador tiene como propósito *definir una dependencia de uno a muchos entre objetos, de manera que cuando el estado de uno cambie se notifique y actualicen automáticamente todos los objetos que dependen de él*. Ambas condiciones se dan aquí: una noticia publicada por un reportero debe llegar a un número variable de canales que el reportero no necesita conocer. La correspondencia elegida es:

- **Sujeto = reportero:** cada uno de los 4 reporteros mantiene su propia lista de suscriptores y, al publicar una noticia, los notifica.
- **Observador = canal de salida:** Web, TV Channel y notificación a la app móvil implementan una misma interfaz y reaccionan cada uno a su manera.

¿Por qué cada reportero es un sujeto y no existe un único sujeto «Redacción»? El enunciado muestra que la noticia del Reportero 1 llega a las tres salidas, mientras que la del Reportero 3 llega únicamente a la Web. Eso implica suscripciones *por reportero*. Con un único sujeto central habría que filtrar por origen dentro de cada observador, trasladando lógica de selección a todos los canales.

Ficha del patrón (los cuatro elementos que definen un patrón de diseño: nombre, problema, solución y consecuencias):

- **Nombre:** Observador (*Observer*), patrón de comportamiento del catálogo GoF, conocido también como *Dependents* o *Publish-Subscribe*.
- **Problema:** notificar un cambio de estado de un objeto a un conjunto variable de objetos interesados sin acoplar al emisor con las clases concretas de los receptores. Aquí, noticias de distintos reporteros que deben llegar a canales de naturaleza diferente (TV, Web, App) y que pueden ampliarse.
- **Solución:** una interfaz de sujeto (`Sujeto`) declara las operaciones de suscripción y notificación; una interfaz de observador (`Observador`) declara `actualizar()`; el sujeto concreto (`Reportero`) invoca `actualizar()` en todos sus suscritos cuando cambia su estado.
- **Consecuencias:** sujetos y observadores varían de forma independiente y las notificaciones se difunden automáticamente; a cambio, pueden producirse actualizaciones inesperadas en cascada y el sujeto no controla qué hace cada observador con la notificación.

Analogía con el ejemplo visto en clase: en el Tema 10 se actualiza en tiempo real la visualización de un catálogo de vehículos: el *Vehículo* es el sujeto y cada *VistaVehículo* es un observador que se actualiza cuando cambia su estado. Nuestro `Reportero` equivale al vehículo y nuestros canales equivalen a las vistas; en ambos casos hay varias «vistas» simultáneas del mismo cambio.

2.3 Ventajas

A continuación se presentan las principales ventajas del patrón Observer en el sistema de noticias. Las tres primeras corresponden a las ventajas señaladas en el Tema 10; las restantes se derivan de la implementación y de las pruebas realizadas.

- **Sujetos y observadores se modifican de forma independiente:** se pueden añadir o cambiar canales sin tocar a los reporteros, y viceversa. En la evolución de la sección 4.1 el archivo `Reportero.java` permaneció idéntico byte a byte.
- **Acoplamiento mínimo:** el sujeto solo sabe que tiene una lista de objetos que cumplen la interfaz `Observador`. Una prueba automática (T12, sección 4.2) verifica que el paquete `sujeto` no importa ninguna clase concreta de canal.
- **Comunicación por difusión:** con una única llamada `notificar()` se avisa a todos los suscritos sin indicar destinatarios concretos; añadir o quitar observadores es trivial.
- **Suscripción dinámica:** `registrar()` y `eliminar()` permiten altas y bajas en tiempo de ejecución, algo imposible en la solución sin patrón.
- **Reutilización:** una misma clase `Reportero` sirve para los 4 reporteros y una misma instancia de canal puede observar a varios reporteros (prueba T07).
- **Facilidad de prueba:** al depender de una interfaz, los canales se pueden sustituir por dobles de prueba (prueba T08 y T09 usan observadores anónimos).

2.4 Inconvenientes

Aplicar el patrón tampoco es gratuito. Se identifican los siguientes inconvenientes, algunos de los cuales se comprobaron experimentalmente en el capítulo 4:

- **Actualizaciones inesperadas (en cascada):** es el inconveniente señalado en el Tema 10: un cambio aparentemente inofensivo en el sujeto puede desencadenar una serie de actualizaciones en los observadores y en los objetos que dependen de ellos. En este sistema el riesgo es bajo porque los canales solo imprimen, pero crecería si un observador volviera a publicar noticias.
- **Un observador defectuoso puede afectar a los demás:** en la etapa 2, sin ningún tratamiento, una excepción en un canal interrumpe la difusión al resto. La prueba T09 lo demuestra (falla en la etapa 2 y pasa en la etapa 3, que aísla cada llamada con try/catch).
- **Orden de notificación no garantizado por contrato:** la implementación manual notifica en orden de suscripción, pero el diseño no debe depender de ello; la propia clase java.util.Observable documenta que su orden es no especificado.
- **Riesgo de fugas de memoria (*lapsed listener*):** si un observador no se da de baja, el sujeto conserva su referencia y no se libera.
- **Notificación síncrona:** un canal lento (por ejemplo, el envío de una notificación *push*) bloquea la publicación; en un sistema real se resolvería con colas o hilos.
- **Mayor cantidad de código y de clases:** 79 líneas en el núcleo frente a 49 de la versión sin patrón (Tabla 1); el costo se justifica solo si se espera evolución.

2.5 Alternativas consideradas

El Tema 10 cierra con una discusión sobre los patrones de comportamiento y destaca cuatro ideas: encapsular lo que varía, usar objetos como argumentos, elegir entre comunicación encapsulada o distribuida, y desacoplar emisores y receptores. El requisito central de este sistema es el último: *difundir un cambio de un emisor a varios receptores desacoplados y variables*. La Tabla 2 contrasta los patrones de comportamiento estudiados en los Temas 8, 9 y 10 con ese requisito.

Tabla 2. Patrones de comportamiento estudiados y su encaje con el sistema de noticias.

Patrón	Finalidad (Temas 8 a 10)	¿Encaja en este sistema?
Observer	Notifica los cambios de estado de un objeto a los suscritos	**Sí.** Relación 1 a N, suscripción dinámica y notificación automática
Mediator	Coordina las relaciones entre sus asociados sin acoplarlos	Posible, pero centraliza la lógica y es innecesario: el flujo es unidireccional (reportero → canal)
Strategy	Selecciona un algoritmo entre varios intercambiables	No. Elige *un* algoritmo; aquí hay que notificar a *todos* los canales suscritos
Command	Encapsula una petición como un objeto (deshacer, colas)	No. No se requiere parametrizar, encolar ni deshacer peticiones
Chain of Responsibility	Varios objetos pueden atender una petición pasándola en cadena	No. La noticia no la atiende «el primero que pueda»: la reciben todos
Iterator	Recorre una colección sin exponer su estructura	Solo como detalle interno (recorrer la lista de observadores)
State	El comportamiento cambia según el estado interno	No. El reportero no cambia de comportamiento
Memento, Interpreter, Template Method, Visitor	Guardar estados, interpretar gramáticas, plantillas con herencia, operar sobre estructuras heterogéneas	No. Ningún requisito del enunciado coincide con su finalidad

Los patrones más cercanos son **Mediator** y **Strategy**. El Mediator sería útil si los canales necesitaran coordinarse entre sí, algo que el enunciado no pide; el Tema 10 lo menciona como patrón relacionado cuando las

actualizaciones son complejas. El Strategy permitiría cambiar *cómo* se emite una noticia en un canal, pero no resuelve la difusión a varios destinos. Por ello se selecciona Observer, que además es un patrón basado en **composición de objetos** (Tema 8, Tabla 1), lo que favorece el desacoplamiento frente a los patrones basados en herencia (Interpreter y Template Method).

2.6 Reutilización de software en la solución

Como se trata de la asignatura Reutilización de Software, conviene identificar qué se reutiliza y a qué nivel:

- **Reutilización de diseño:** el patrón Observer es una solución probada para un problema recurrente (Gamma, 2002); se reutiliza su estructura, no su código.
- **Reutilización de clases:** una sola clase `Reportero` se instancia 4 veces, y cada canal es una única clase usable por cualquier reportero.
- **Reutilización de la plataforma:** el JDK ya ofrece mecanismos del patrón; `java.util.Observable` está obsoleta desde Java 9 y `java.beans.PropertyChangeSupport` es la alternativa vigente, implementada y comparada en la sección 4.3.
- **Reutilización de pruebas:** la misma suite de 12 pruebas se ejecutó contra dos etapas distintas del código (sección 4.2).

3. EJERCICIO 2: DISEÑO E IMPLEMENTACIÓN

En este ejercicio se presenta el diseño de la solución mediante diagramas UML y su implementación en Java, con **4 reporteros y 3 tipos de salida** (Web, TV Channel y notificación a la app móvil). Se explica además por qué se emplea Observer en lugar de los demás patrones de comportamiento estudiados, conforme a la Tabla 2 del apartado 2.5.

3.1 Participantes del patrón

El diseño sigue los cuatro elementos del patrón descritos en el Tema 10 (interfaz sujeto, sujeto concreto, interfaz observador y observador concreto), más una clase de datos `Noticia` que viaja del sujeto al observador. La Tabla 3 resume la responsabilidad de cada participante y la Tabla 4 las suscripciones configuradas en la simulación.

Tabla 3. Participantes del patrón Observer en el sistema de noticias.

Elemento del patrón	Clase / interfaz	Responsabilidad
Interfaz Sujeto	Sujeto	Declara registrar(), eliminar() y notificar()
Sujeto concreto	Reportero	Guarda su lista de observadores y la última noticia; publicarNoticia() cambia el estado y notifica
Interfaz Observador	Observador	Declara actualizar(Noticia) y getNombre()
Observador concreto	CanalWeb, CanalTV, NotificacionApp	Cada uno reacciona a su manera ante la noticia recibida
Dato transportado	Noticia (record)	Contiene reportero y titular; es inmutable
Cliente / composición	Main	Crea 4 reporteros y 3 canales, configura las suscripciones y ejecuta la simulación

Tabla 4. Suscripciones configuradas en la simulación (✓= el canal recibe las noticias del reportero).

Reportero	Web	TV Channel	App móvil
Reportero 1	✓	✓	✓
Reportero 2		✓	✓
Reportero 3	✓		
Reportero 4	✓	✓	

Con estas suscripciones se reproduce el comportamiento del enunciado: la noticia del Reportero 1 llega a las tres salidas y la del Reportero 3 solo a la Web. Las suscripciones de los reporteros 2 y 4 no están fijadas por el enunciado, por lo que se definieron para ejercitar los demás casos posibles.

3.2 Diagrama UML del sistema

A continuación se presenta el diagrama de clases correspondiente al diseño del sistema (Figura 2). Se observa que Reportero implementa Sujeto y mantiene una agregación de cero o más Observador; los tres canales implementan Observador y reciben una Noticia. Reportero solo depende de la interfaz, nunca de las clases concretas de los canales.

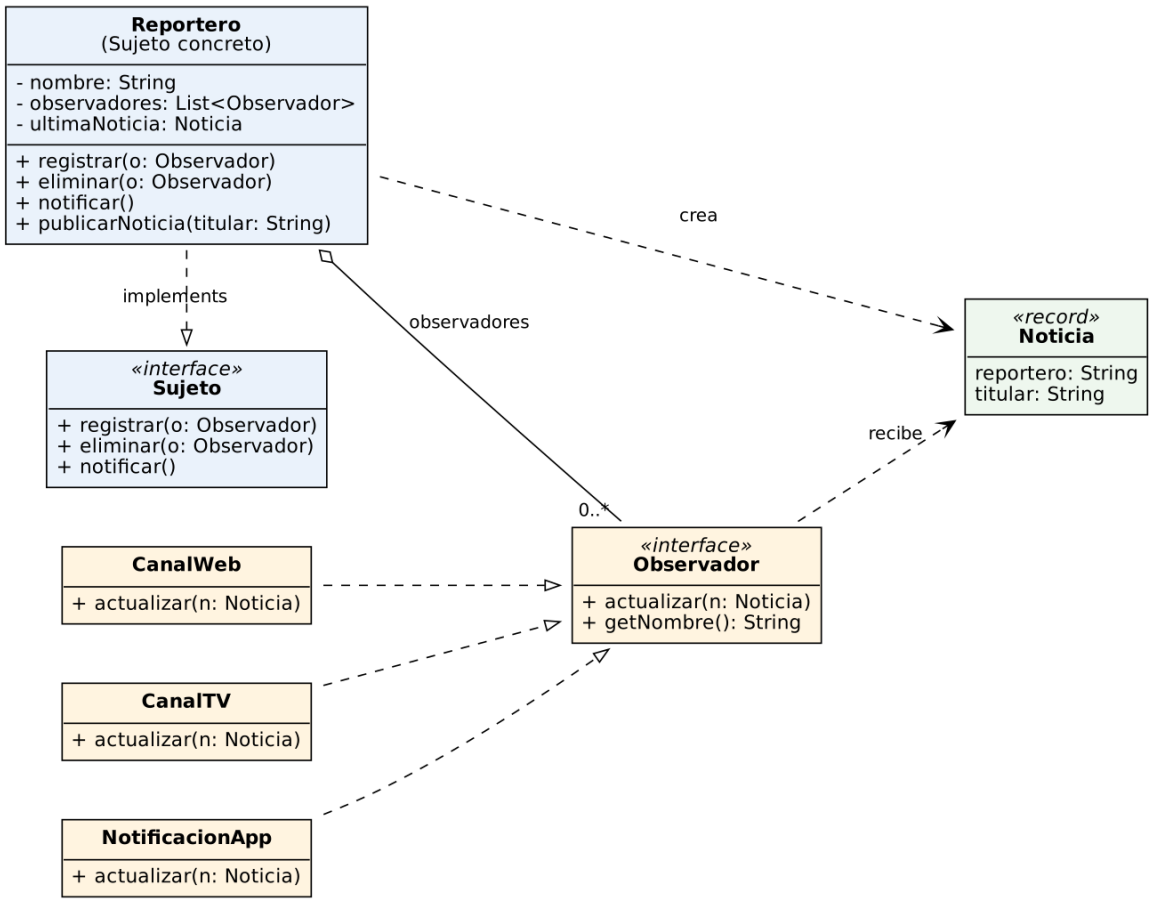


Figura 2. Diagrama de clases UML del sistema de noticias con el patrón Observer.

La Figura 3 resume la misma idea desde la perspectiva de las dependencias: los cuatro reporteros dependen únicamente de la interfaz Observador, que implementan los tres canales. Compárese con la Figura 1, donde existen 8 dependencias directas hacia clases concretas.

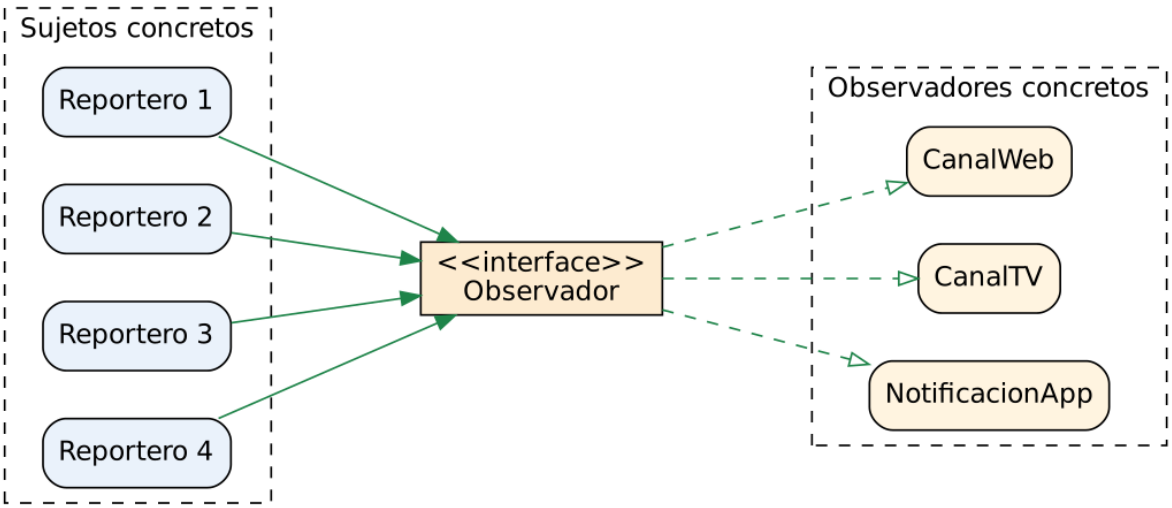


Figura 3. Solución con Observer: los 4 reporteros dependen solo de la interfaz Observador, implementada por los 3 canales.

Finalmente, el diagrama de secuencia de la Figura 4 muestra el flujo dinámico cuando el Reportero 1 publica una noticia: publicarNoticia() cambia el estado, invoca notificar() y este recorre la lista de observadores llamando a actualizar() en cada uno.

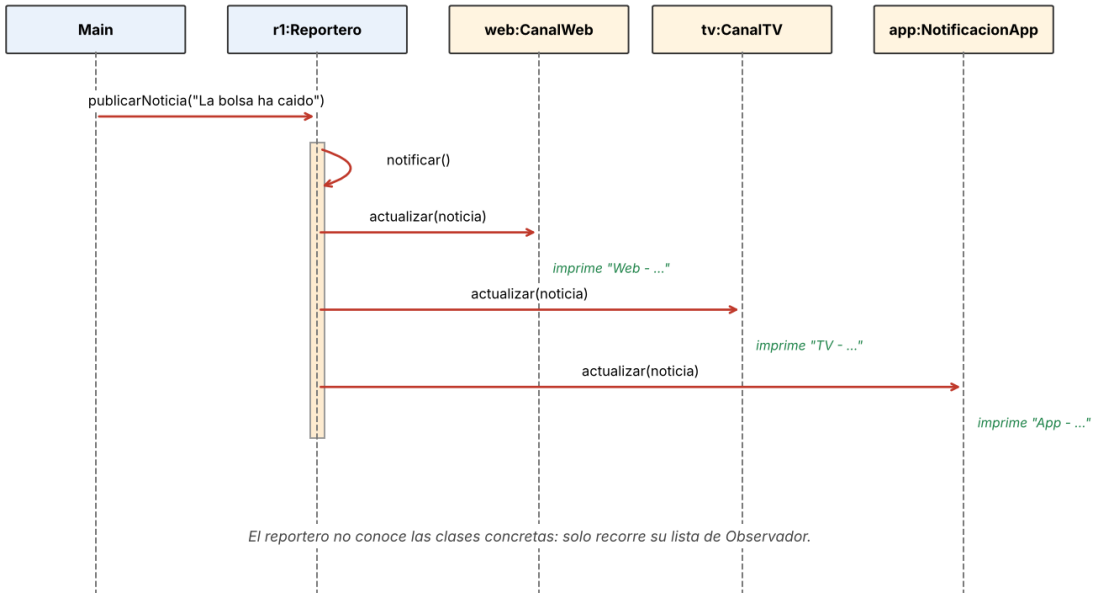


Figura 4. Diagrama de secuencia: el Reportero 1 publica una noticia y se notifica a Web, TV y App.

3.3 Implementación en Java

La implementación del patrón se desarrolla en Java 16 o superior (se usa un *record* para la noticia). Para mantener la independencia entre responsabilidades, el proyecto se estructura en paquetes bajo etapa1_base/src/com/unir/noticias:

- **Paquete modelo:** contiene Noticia, un *record* inmutable con el reportero y el titular.
- **Paquete observador:** contiene la interfaz Observador y los tres observadores concretos CanalWeb, CanalTV y NotificacionApp.

- **Paquete sujeto:** contiene la interfaz Sujeto y su implementación concreta Reportero. Este paquete no importa ninguna clase de canal.
- **Clase Main:** punto de composición. Crea los objetos, configura las suscripciones y ejecuta la simulación, en modo interactivo (entrada por teclado) o en modo --demo.

A continuación se muestra el código. Primero el dato transportado y la interfaz del observador:

```
package com.unir.noticias.modelo;

/**
 * Noticia inmutable que viaja del sujeto (reportero) hacia los observadores.
 * Se usa un record (Java 16+) para evitar codigo repetitivo.
 */
public record Noticia(String reportero, String titular) {
    @Override
    public String toString() {
        return reportero + " - " + titular;
    }
}
```

```
package com.unir.noticias.observador;

import com.unir.noticias.modelo.Noticia;

/** Interfaz Observador: contrato que cumple cualquier canal que desee recibir noticias. */
public interface Observador {

    /** Metodo de actualizacion invocado por el sujeto cuando publica una noticia. */
    void actualizar(Noticia noticia);

    /** Nombre del canal (Web, TV, App); util para trazas y para la suscripcion. */
    String getNombre();
}
```

Un observador concreto (CanalTV y NotificacionApp son análogos: solo cambian el prefijo impreso y el nombre):

```
package com.unir.noticias.observador;

import com.unir.noticias.modelo.Noticia;

/** Observador concreto 1: publica la noticia en la pagina web. */
public class CanalWeb implements Observador {

    @Override
    public void actualizar(Noticia noticia) {
        System.out.println("Web - " + noticia.titular());
    }

    @Override
    public String getNombre() {
        return "Web";
    }
}
```

La interfaz del sujeto y su implementación concreta, el reportero:

```
package com.unir.noticias.sujeto;

import com.unir.noticias.observador.Observador;

/** Interfaz Sujeto: gestiona la lista de observadores y la notificacion. */
public interface Sujeto {
    void registrar(Observador observador);
    void eliminar(Observador observador);
    void notificar();
}
```

```java
package com.unir.noticias.sujeto;

import com.unir.noticias.modelo.Noticia;
import com.unir.noticias.observador.Observador;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Sujeto concreto: cada reportero mantiene su propia lista de observadores
 * suscritos y, al publicar una noticia, los notifica a todos (difusion).
 */
public class Reportero implements Sujeto {

    private final String nombre;
    private final List<Observador> observadores = new ArrayList<>();
    private Noticia ultimaNoticia;

    public Reportero(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void registrar(Observador observador) {
        if (observador != null && !observadores.contains(observador)) {
            observadores.add(observador);
        }
    }

    @Override
    public void eliminar(Observador observador) {
        observadores.remove(observador);
    }

    @Override
    public void notificar() {
        // Copia defensiva: un observador podria darse de baja durante la notificacion.
        for (Observador o : new ArrayList<>(observadores)) {
            o.actualizar(ultimaNoticia);
        }
    }

    /** Cambio de estado del sujeto: nueva noticia -> dispara la notificacion. */
    public void publicarNoticia(String titular) {
        this.ultimaNoticia = new Noticia(nombre, titular);
        notificar();
    }

    public String getNombre() {
        return nombre;
    }

    public List<Observador> getObservadores() {
        return Collections.unmodifiableList(observadores);
    }
}
```

Finalmente, la clase Main, que construye el escenario de 4 reporteros y 3 canales y ejecuta la simulación:

```
package com.unir.noticias;

import com.unir.noticias.observador.CanalTV;
import com.unir.noticias.observador.CanalWeb;
import com.unir.noticias.observador.NotificacionApp;
import com.unir.noticias.observador.Observador;
import com.unir.noticias.sujeto.Reportero;

import java.util.List;
import java.util.Scanner;

/**
 * Simulacion del sistema de noticias con el patron Observer.
 *  - Sin argumentos  : modo interactivo (entrada por teclado).
 *  - Argumento --demo: escenario de ejemplo del enunciado.
 */
public class Main {

    public static void main(String[] args) {
        // 3 observadores concretos (tipos de salida)
        Observador web = new CanalWeb();
        Observador tv = new CanalTV();
        Observador app = new NotificacionApp();

        // 4 sujetos concretos (reporteros)
        Reportero r1 = new Reportero("Reportero 1");
        Reportero r2 = new Reportero("Reportero 2");
        Reportero r3 = new Reportero("Reportero 3");
        Reportero r4 = new Reportero("Reportero 4");

        // Suscripciones: cada canal decide que reporteros le interesan
        for (Observador o : List.of(web, tv, app)) r1.registrar(o); // R1 -> Web, TV, App
        r2.registrar(tv);  r2.registrar(app);                       // R2 -> TV, App
        r3.registrar(web);                                          // R3 -> Web
        r4.registrar(web); r4.registrar(tv);                        // R4 -> Web, TV

        List<Reportero> reporteros = List.of(r1, r2, r3, r4);

        if (args.length > 0 && args[0].equals("--demo")) {
            demo(r1, r2, r3, r4);
        } else {
            interactivo(reporteros);
        }
    }

    private static void demo(Reportero r1, Reportero r2, Reportero r3, Reportero r4) {
        emitir(r1, "La bolsa ha caído");
        emitir(r3, "El Real Madrid de nuevo gana la Champions");
        emitir(r2, "Nueva actualización de la app disponible");
        emitir(r4, "Lluvias intensas en Bogotá esta semana");
    }

    private static void emitir(Reportero r, String titular) {
        System.out.println("Esperando noticia: " + r.getNombre() + " - " + titular);
        r.publicarNoticia(titular);
        System.out.println();
    }

    private static void interactivo(List<Reportero> reporteros) {
        Scanner in = new Scanner(System.in);
        System.out.println("=== Sistema de noticias (Patron Observer) ===");
        for (int i = 0; i < reporteros.size(); i++) {
            Reportero r = reporteros.get(i);
            StringBuilder canales = new StringBuilder();
            for (Observador o : r.getObservadores()) canales.append(o.getNombre()).append(' ');
            System.out.println((i + 1) + ") " + r.getNombre() + "  [suscritos: " +
canales.toString().trim() + "]");
        }
        System.out.println("Escriba: <n reportero> <titular>   (ej. 1 La bolsa ha caído)   |   'salir' para
terminar");

        while (true) {
            System.out.print("> ");
            if (!in.hasNextLine()) break;
            String linea = in.nextLine().trim();
            if (linea.equalsIgnoreCase("salir")) break;
            if (linea.isEmpty()) continue;

            String[] partes = linea.split("\\s+", 2);
            try {
                int idx = Integer.parseInt(partes[0]) - 1;
                if (partes.length < 2 || idx < 0 || idx >= reporteros.size()) throw new
IllegalArgumentException();
```

```
                emitir(reporteros.get(idx), partes[1]);
            } catch (IllegalArgumentException e) {
                System.out.println("Entrada no valida. Formato: <1-4> <titular>");
            }
        }
        System.out.println("Fin de la simulacion.");
    }
}
```

3.4 Decisiones de diseño

- **Interfaces en lugar de clase abstracta para el sujeto:** el ejemplo del Tema 10 usa una clase abstracta `Sujeto`. Aquí se prefiere una interfaz más una clase concreta que usa composición (una lista), porque no consume la única herencia de Java y es más fácil de probar.
- **Implementación manual y no `java.util.Observable`:** la clase del JDK está obsoleta desde Java 9, obliga a heredar de ella y su orden de notificación no está especificado; la versión manual cumple el patrón con menos acoplamiento al JDK.
- **Modelo *push*:** el sujeto envía la noticia completa en `actualizar(Noticia)`, de modo que el observador no necesita consultar al sujeto. Es lo adecuado porque la noticia es pequeña e inmutable.
- **`Noticia` como *record* inmutable:** se comparte entre varios observadores sin riesgo de que uno la altere para los demás.
- **Copia defensiva al notificar:** `notificar()` itera sobre una copia de la lista, de modo que un observador pueda darse de baja durante la notificación sin provocar una excepción de modificación concurrente (prueba T08).
- **Registro sin duplicados:** `registrar()` ignora `null` y observadores ya suscritos, evitando notificaciones repetidas (pruebas T04 y T05).
- **Salida por consola:** los canales imprimen con `System.out.println` tal como indica el enunciado; en un sistema real cada uno invocaría su servicio (CMS, emisión, *push*) manteniendo la misma interfaz.

3.5 Resultados de la ejecución

El sistema se compiló con `javac` sin errores ni advertencias y se ejecutó en sus dos modos. En el modo de demostración (`--demo`) se emiten noticias de los cuatro reporteros (Figura 5).

```
Terminal - etapa1_base (--demo)
$ java -cp out com.unir.noticias.Main --demo
Esperando noticia: Reportero 1 - La bolsa ha caído
Web - La bolsa ha caído
TV - La bolsa ha caído
App - La bolsa ha caído

Esperando noticia: Reportero 3 - El Real Madrid de nuevo gana la Champions
Web - El Real Madrid de nuevo gana la Champions

Esperando noticia: Reportero 2 - Nueva actualización de la app disponible
TV - Nueva actualización de la app disponible
App - Nueva actualización de la app disponible

Esperando noticia: Reportero 4 - Lluvias intensas en Bogotá esta semana
Web - Lluvias intensas en Bogotá esta semana
TV - Lluvias intensas en Bogotá esta semana
```

Figura 5. Ejecución del escenario de ejemplo (modo `--demo`).

Análisis del resultado. La noticia del Reportero 1 llega a las tres salidas y la del Reportero 3, únicamente a la Web, tal como se plantea en el enunciado. Los reporteros 2 y 4 confirman que cada sujeto notifica solo a sus propios suscritos.

En el modo interactivo, el usuario escribe el número de reportero seguido del titular; el sistema valida la entrada, notifica a los observadores suscritos y rechaza las entradas no válidas (Figura 6). Se escribe `salir` para terminar.

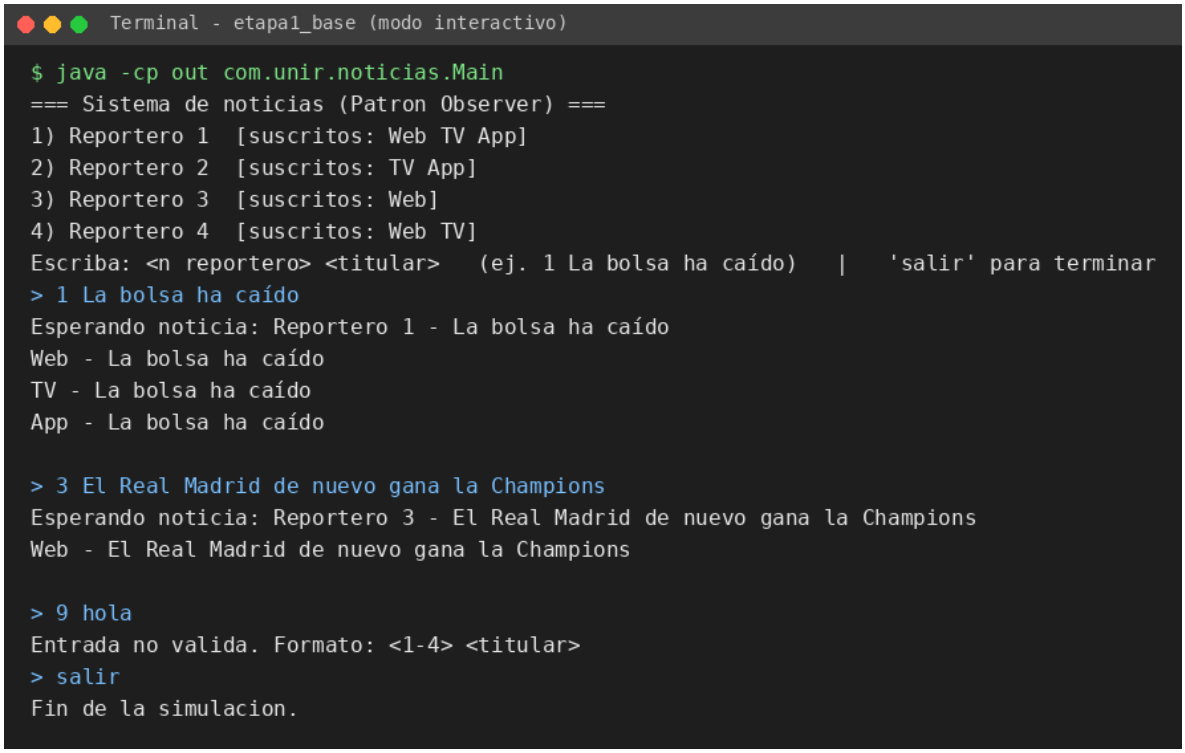


Figura 6. Ejecución en modo interactivo: entrada por teclado, notificaciones y validación de una entrada incorrecta.

4. DESARROLLO ADICIONAL: MÁS ALLÁ DEL ENUNCIADO

Este capítulo amplía voluntariamente el alcance de la actividad con evidencia técnica: se evoluciona el sistema con un nuevo canal y se mide el costo frente a la versión sin patrón, se verifica el comportamiento con pruebas automatizadas (incluida una de arquitectura) y se implementa una alternativa basada en el JDK. Cada etapa es independiente y tiene su propio `main`.

4.1 Evolución del sistema: nuevo canal Email

Para comprobar la extensibilidad se añade un cuarto observador, `CanalEmail`, suscrito a los reporteros 1 y 4 (etapa 2). El diagrama UML ampliado (Figura 7) muestra la única clase nueva; el resto del diseño permanece igual.

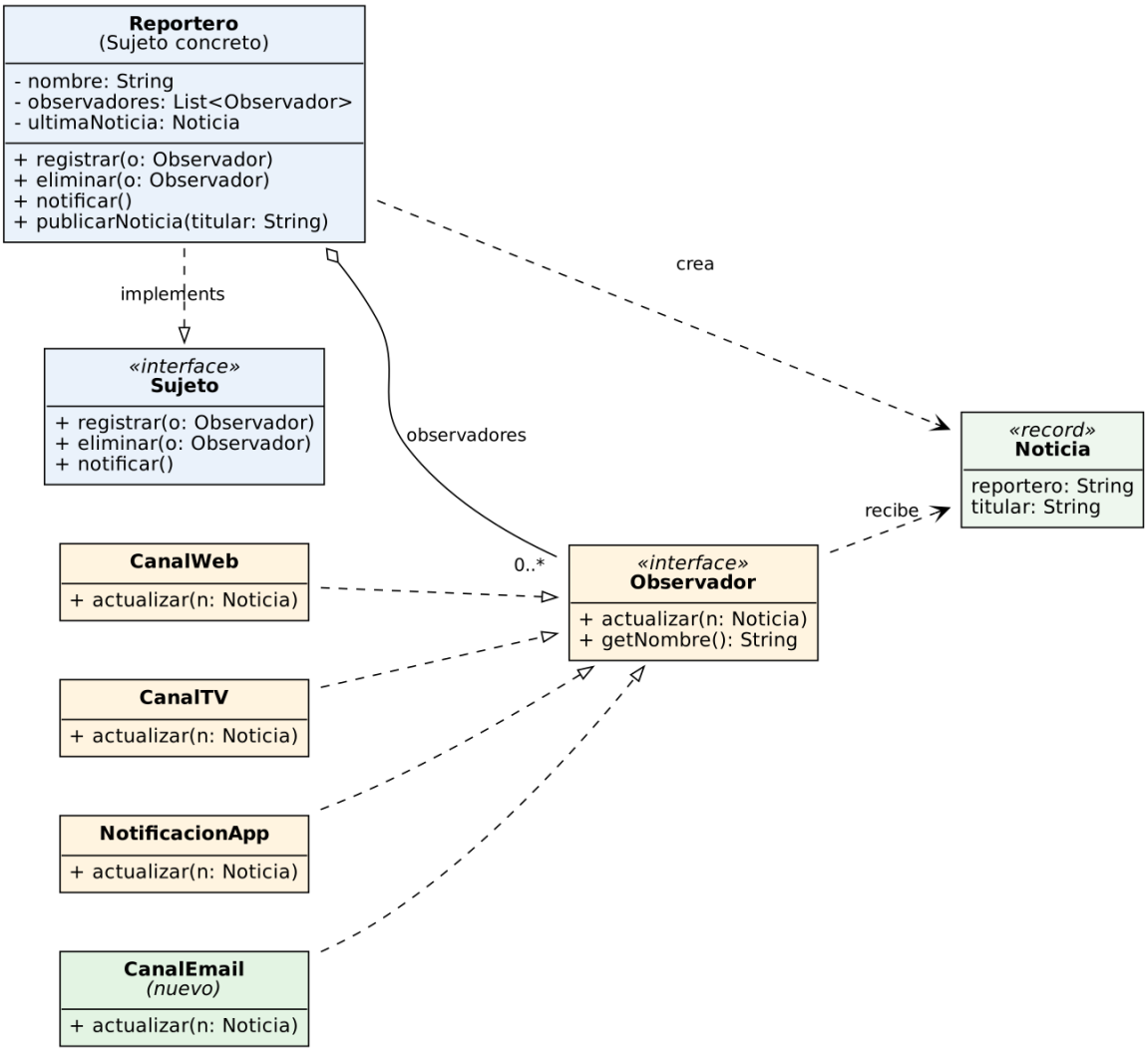


Figura 7. Diagrama UML ampliado: se añade CanalEmail (en verde) sin modificar Sujeto, Reportero ni los demás canales.

El cambio completo consiste en una clase nueva y tres líneas en Main:

```
package com.unir.noticias.observador;

import com.unir.noticias.modelo.Noticia;

/** Observador concreto 4 (evolucion): envia la noticia por correo electronico. */
public class CanalEmail implements Observador {

    @Override
    public void actualizar(Noticia noticia) {
        System.out.println("Email - " + noticia.titular());
    }

    @Override
    public String getNombre() {
        return "Email";
    }
}
```

```
// Main.java (etapa 2): lineas anadidas
import com.unir.noticias.observador.CanalEmail;
...
Observador email = new CanalEmail();          // EVOLUCION: nuevo canal
...
r1.registrar(email); r4.registrar(email);     // EVOLUCION: R1 y R4 -> Email
```

El archivo Reportero.java quedó **sin ningún cambio** (se verificó con diff). La Figura 8 muestra la nueva salida: las noticias de los reporteros 1 y 4 incluyen ahora la línea de Email. La versión sin patrón equivalente exigió modificar Reportero1 y Reportero4, añadir un campo y una llamada en cada uno, y su salida es idéntica a la de la etapa 2 (Tabla 1).

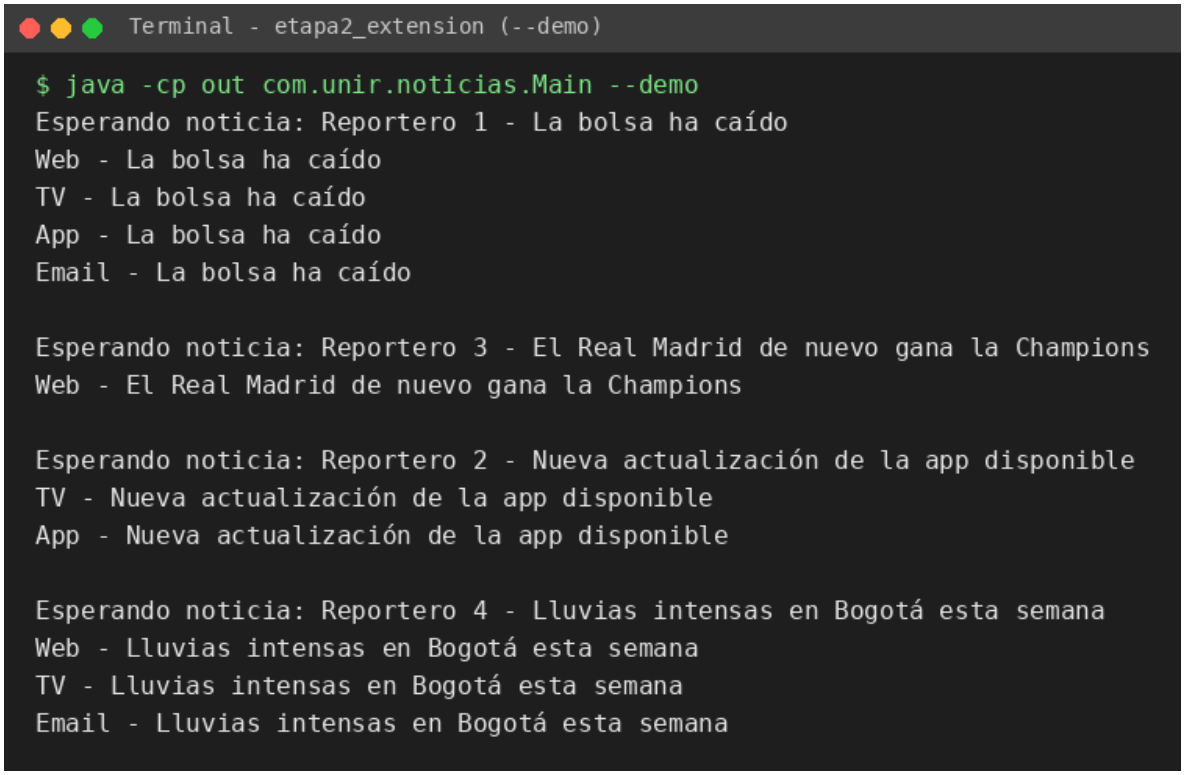


Figura 8. Ejecución de la etapa 2: los reporteros 1 y 4 notifican también por Email.

Conviene matizar el resultado: Observer **no elimina** el trabajo de decidir qué canal escucha a qué reportero —cada suscripción sigue siendo una línea de código—, pero lo concentra en un único punto de composición y evita tocar las clases de negocio. Con la versión sin patrón, el número de archivos modificados crece con la cantidad de reporteros que deban usar el nuevo canal.

4.2 Pruebas automatizadas, incluida una prueba de arquitectura

La etapa 3 incorpora una suite de 12 pruebas sin dependencias externas (95 líneas en PruebasObserver.java; no requiere JUnit). Las pruebas capturan lo que cada canal imprime y verifican el comportamiento del patrón. La Tabla 5 las describe.

Tabla 5. Suite de pruebas de la etapa 3.

Prueba	Qué verifica
T01	El Reportero 1 notifica a Web, TV y App en orden de suscripción
T02	Un reportero solo notifica a sus suscritos (solo Web)
T03	Tras eliminar(), el canal deja de recibir noticias
T04	Registrar dos veces el mismo observador no duplica la notificación
T05	Registrar null se ignora sin lanzar excepción
T06	Sin observadores, publicar no falla ni imprime
T07	Un mismo canal puede observar a varios reporteros
T08	La baja de un observador durante la notificación no rompe la iteración
T09	Un observador que falla no impide notificar a los demás

Prueba	Qué verifica
T10	Extensibilidad: un canal nuevo recibe noticias sin tocar Reportero
T11	Noticia es inmutable y conserva reportero y titular
T12	**Arquitectura:** el paquete sujeto no depende de ningún canal concreto

Para llegar a la prueba T09 fue necesario modificar el sujeto: en la etapa 3 Reportero.notificar() envuelve cada llamada en un try/catch que registra el error y continúa con el siguiente observador. Es la mitigación del inconveniente descrito en la sección 2.4:

```
// Reportero.java (etapa 3)
for (Observador o : new ArrayList<>(observadores)) {
    try {
        o.actualizar(ultimaNoticia);
    } catch (RuntimeException e) {
        // Aislamiento de fallos: un canal defectuoso no impide avisar a los demas
        System.err.println("[ERROR] " + o.getNombre() + " fallo: " + e.getMessage());
    }
}
```

La prueba de arquitectura (T12) lee el código fuente del paquete sujeto y comprueba que no aparece ninguno de los nombres de canales concretos. La Figura 9 muestra la ejecución de la etapa 3: 12 de 12 pruebas superadas.

```
Terminal - etapa3_pruebas

$ java -cp out com.unir.noticias.PruebasObserver src
=== Pruebas del patron Observer ===
  [OK]    T01 R1 notifica a Web, TV y App en orden de suscripcion
  [OK]    T02 Un reportero solo notifica a sus suscritos (solo Web)
  [OK]    T03 Tras eliminar() un canal deja de recibir noticias
  [OK]    T04 Registrar dos veces el mismo observador no duplica la notificacion
  [OK]    T05 Registrar null se ignora sin lanzar excepcion
  [OK]    T06 Sin observadores, publicar no falla ni imprime
  [OK]    T07 Un mismo canal observa a varios reporteros
  [OK]    T08 Baja de un observador durante la notificacion no rompe la iteracion
  [OK]    T09 Un observador que falla no impide notificar a los demas
  [OK]    T10 Extensibilidad: un canal nuevo recibe noticias sin tocar Reportero
  [OK]    T11 La Noticia es inmutable y conserva reportero y titular
  [OK]    T12 ARQUITECTURA: el paquete sujeto no depende de canales concretos

Resultado: 12/12 pruebas superadas
```

Figura 9. Ejecución de la suite de la etapa 3: 12 de 12 pruebas superadas.

Una prueba solo es útil si puede fallar, por lo que se comprobaron dos escenarios negativos. En el primero, se introdujo una violación arquitectónica deliberada: se añadió a Reportero un import de CanalWeb. La prueba T12 la detectó y la suite pasó a 11 de 12 (Figura 10). En el segundo, se ejecutó la misma suite contra el Reportero de la etapa 2, que no aísla excepciones: la prueba T09 falló, lo que confirma el inconveniente descrito en la sección 2.4 (Figura 11).

```
Terminal - violacion arquitectonica deliberada

$ java -cp out com.unir.noticias.PruebasObserver src   # Reportero importa CanalWeb
  [FALLO] T12 ARQUITECTURA: el paquete sujeto no depende de canales concretos
Resultado: 11/12 pruebas superadas
Fallos: [T12 ARQUITECTURA: el paquete sujeto no depende de canales concretos]
```

Figura 10. Violación arquitectónica deliberada (Reportero importa CanalWeb): la prueba T12 la detecta.

```
Terminal - suite sobre etapa2 (sin try/catch)

$ java -cp out com.unir.noticias.PruebasObserver src   # Reportero de la etapa 2
  [FALLO] T09 Un observador que falla no impide notificar a los demas
Resultado: 11/12 pruebas superadas
Fallos: [T09 Un observador que falla no impide notificar a los demas]
```

Figura 11. Suite aplicada al Reportero de la etapa 2 (sin try/catch): la prueba T09 falla.

4.3 Alternativa implementada: PropertyChangeSupport

El JDK incluye un mecanismo del patrón Observer que no está obsoleto: java.beans.PropertyChangeSupport. En la etapa 4 se reimplementa el mismo escenario con esta clase; los canales se reducen a *lambdas* porque PropertyChangeListener es una interfaz funcional:

```
package com.unir.noticias;

import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

/** Alternativa: el JDK ya trae un mecanismo Observer (java.beans.PropertyChangeSupport). */
public class Reportero {
    private final String nombre;
    private final PropertyChangeSupport soporte = new PropertyChangeSupport(this);

    public Reportero(String nombre) { this.nombre = nombre; }

    public void suscribir(PropertyChangeListener oyente) { soporte.addPropertyChangeListener(oyente); }
    public void baja(PropertyChangeListener oyente)      { soporte.removePropertyChangeListener(oyente); }

    public void publicarNoticia(String titular) {
        // oldValue = null evita que dos titulares iguales consecutivos se descarten como "sin cambio"
        soporte.firePropertyChange(new PropertyChangeEvent(this, "noticia", null, titular));
    }

    public String getNombre() { return nombre; }
}
```

La salida de esta versión es idéntica a la de la etapa 1, algo que se comprobó con diff junto con la de la versión sin patrón (Figura 12).

```
Terminal - comparación de salidas

$ diff salida_sinpatron.txt salida_observer.txt
diff sin patrón vs Observer: exit=0 (salidas idénticas)
diff PropertyChangeSupport vs Observer: exit=0 (salidas idénticas)
```

Figura 12. Comparación de salidas: la versión sin patrón y la basada en PropertyChangeSupport producen exactamente la misma salida que el Observer manual.

Tabla 6. Observer manual frente a PropertyChangeSupport (medido sobre etapa1_base/ y etapa4_alternativa/).

Aspecto	Observer manual (etapa 1)	PropertyChangeSupport (etapa 4)
Archivos Java	8	2
Líneas del núcleo (sin Main)	79	11
Canales	Clases que implementan Observador	Lambdas o clases PropertyChangeListener
Control total del contrato	Sí (actualizar(Noticia) tipado)	Menos: el evento es genérico (getNewValue() devuelve Object)
Aislamiento de excepciones	Se añadió explícitamente (etapa 3)	No lo incluye por defecto
Dependencia del JDK	Ninguna	java.desktop (módulo java.beans)

Se encontró además un matiz importante en esta clase: firePropertyChange **no notifica** si el valor anterior y el nuevo son iguales y no nulos. Con la noticia como valor, dos titulares idénticos consecutivos se perderían. Por eso publicarNoticia() crea el evento con oldValue = null, lo que se verificó con un programa aparte. La alternativa es más breve, pero ese comportamiento implícito es un riesgo que el Observer manual no tiene.

4.4 Guía de decisión y recomendación

A partir de los resultados se propone la siguiente guía para elegir entre las opciones evaluadas (Tabla 7):

Tabla 7. Guía de decisión según el escenario.

Escenario	Recomendación	Razón
Pocos canales fijos que casi no cambian	Llamada directa (sin patrón)	Menos código y menos clases; el costo del patrón no se amortiza
Canales que se añaden o cambian con frecuencia	**Observer manual**	Cambio localizado: el reportero no se modifica (0 archivos de reportero tocados)
Prototipo rápido o aplicación con interfaz gráfica	PropertyChangeSupport	Mínimo código y ya integrado en el JDK; cuidado con eventos de valor igual
Canales lentos o remotos, gran volumen	Observer con cola o hilos, o un *broker* de mensajería	La notificación síncrona bloquea al sujeto; hace falta desacoplar también en el tiempo
Código nuevo en Java 9 o superior	No usar java.util.Observable	Está obsoleta y no garantiza el orden de notificación

Recomendación para el sistema de noticias: dado que el enunciado ya contempla suscriptores de tipos distintos y es previsible que aparezcan nuevos canales, se recomienda el Observer manual de la etapa 3 (con aislamiento de excepciones y pruebas). Es la opción que protege las clases de negocio frente al cambio, que es el verdadero beneficio del patrón.

5. CONCLUSIONES

El desarrollo de esta actividad permitió analizar, diseñar, implementar y evaluar el patrón de comportamiento Observer, aplicado a un sistema de noticias en Java. A través de los ejercicios y las etapas de evolución fue posible contrastar sus fundamentos teóricos con los resultados obtenidos, identificando tanto sus ventajas estructurales como sus limitaciones prácticas. Asimismo, la comparación con una solución sin patrón y con otros patrones de comportamiento permitió establecer en qué escenarios su utilización representa una solución conveniente.

A partir del diseño, la implementación y las pruebas realizadas, se establecen las siguientes conclusiones:

- **Aplicabilidad del patrón Observer:** resuelve adecuadamente el problema cuando se modela a cada reportero como sujeto y a cada canal como observador. El reportero opera solo a través de la interfaz Observador y no conoce las clases concretas de los canales.

- **Impacto de la evolución:** añadir el canal Email requirió una clase nueva y tres líneas en Main, sin modificar ninguna clase de reportero, mientras que la versión sin patrón obligó a modificar dos clases de reportero.
- **Costo estructural:** el patrón no es gratuito. A este tamaño el núcleo del Observer tiene 79 líneas frente a 49 de la versión sin patrón. Su valor se hace evidente cuando se espera que el sistema evolucione, no en un sistema con pocos canales fijos.
- **Comparación con otros patrones:** Mediator, Strategy, Command y Chain of Responsibility cumplen propósitos diferentes. El requisito de difundir un cambio a *todos* los suscritos solo lo cubre directamente Observer.
- **Inconvenientes comprobados:** el riesgo de que un observador defectuoso afecte a los demás se demostró con la prueba T09, que falla en la etapa 2 y pasa en la etapa 3 tras aislar cada llamada con try/catch.
- **Verificación de la arquitectura:** la prueba T12 comprueba de forma automática que el paquete sujeto no depende de canales concretos y detectó una violación introducida deliberadamente.
- **Alternativas del JDK:** java.util.Observable está obsoleta; PropertyChangeSupport reduce el código a 11 líneas en el núcleo, pero su evento genérico y su descarte de valores iguales exigen precaución.
- **Reutilización y verificación:** la reutilización se abordó desde la separación de responsabilidades y se contrastó con pruebas y comparaciones de salida entre etapas; las tres versiones del sistema producen exactamente la misma salida.

En conclusión, la actividad permitió comprobar que la calidad de una decisión de diseño depende de su correspondencia con el problema y del comportamiento que demuestra frente al cambio. Observer ofrece una estructura sólida para difundir cambios de estado a receptores desacoplados, pero introduce costos que deben evaluarse antes de adoptarlo. No existe un patrón universalmente superior, sino alternativas cuyo valor depende de los requisitos y de la naturaleza de los cambios previstos.

Finalmente, el ejercicio permitió integrar los conceptos de diseño orientado a objetos con prácticas de implementación, pruebas automatizadas, análisis comparativo y evaluación arquitectónica. De esta manera se reafirma que aplicar un patrón no consiste únicamente en reproducir una estructura conocida, sino en comprender su propósito, justificar su utilización, reconocer sus limitaciones y verificar con evidencia técnica que aporta beneficios reales.

6. BIBLIOGRAFÍA

Las siguientes referencias corresponden a las fuentes bibliográficas y materiales académicos consultados para fundamentar el desarrollo de la actividad, en particular los patrones de comportamiento, su comparación y los principios de diseño aplicados.

- Debrauwer, L. (2013). *Patrones de diseño en Java*. Ediciones ENI.
- Gamma, E. (2002). *Patrones de diseño* (2.ª ed.). Pearson Educación.
- Kanasz, R. (2013, 10 de enero). Design Patterns 3 of 3 – Behavioral design patterns. *Code Project*. https://www.codeproject.com/Articles/455228/Design-Patterns-3-of-3-Behavioral-Design-Patterns
- Oracle. (s. f.). Class Observable (Java SE 9 & JDK 9 API). https://docs.oracle.com/javase/9/docs/api/java/util/Observable.html
- Ram N Java. (2013, 14 de septiembre). *Observer design pattern – Real time example* [Vídeo]. YouTube. https://www.youtube.com/watch?v=Ocqj4vyzeQQ
- Source Making. (s. f.). Behavioral patterns. https://sourcemaking.com/design_patterns/behavioral_patterns
- Tedeschi, N. (2008, 14 de octubre). ¿Qué es un patrón de diseño? Microsoft. https://msdn.microsoft.com/es-es/library/bb972240.aspx
- Universidad Internacional de La Rioja. (s. f.). Tema 8. Patrones de comportamiento [Material didáctico]. En *Reutilización de Software*. UNIR.
- Universidad Internacional de La Rioja. (s. f.). Tema 9. Patrones de comportamiento (II) [Material didáctico]. En *Reutilización de Software*. UNIR.

- Universidad Internacional de La Rioja. (s. f.). Tema 10. Patrones de comportamiento (III) [Material didáctico]. En *Reutilización de Software*. UNIR.

7. ANEXO: ENTREGA DEL CÓDIGO Y EJECUCIÓN

Repositorio: https://github.com/AlejoTechEngineer/observer-sistema-noticias. La entrega consiste en este documento y en el enlace al repositorio, donde el código puede copiarse y ejecutarse siguiendo el archivo README. Cada etapa es independiente y tiene su propio main:

```
etapa1_base/        Ejercicio 2: sistema base (4 reporteros x 3 canales)
etapa2_extension/   Cap. 4: + canal Email sin modificar Reportero
etapa3_pruebas/     Cap. 4: aislamiento de excepciones + 12 pruebas (incl. arquitectura)
etapa4_alternativa/ Cap. 4: alternativa con java.beans.PropertyChangeSupport
sinpatron/          Ejercicio 1: version sin patron (y sinpatron_ext/ con el canal Email)
docs/               Diagramas UML (PNG y fuente Graphviz .dot) y capturas
```

Compilación y ejecución (requiere JDK 16 o superior; se probó con JDK 21). Si la consola muestra caracteres extraños en las tildes, se puede añadir -Dstdout.encoding=UTF-8 al comando java:

```
$ cd etapa1_base
$ javac -encoding UTF-8 -d out $(find src -name "*.java")
$ java -cp out com.unir.noticias.Main            # modo interactivo
$ java -cp out com.unir.noticias.Main --demo     # escenario de ejemplo
$ cd ../etapa3_pruebas
$ javac -encoding UTF-8 -d out $(find src test -name "*.java")
$ java -cp out com.unir.noticias.PruebasObserver src    # suite de 12 pruebas
```

8. AGRADECIMIENTO

Finalmente, deseo expresar mi sincero agradecimiento al profesor Deivis Eduard Ramírez Martínez por el acompañamiento y los conocimientos compartidos a lo largo de la asignatura Reutilización de Software.

El desarrollo de esta actividad permitió comprender los patrones de comportamiento no solo como una teoría, sino como una herramienta cuya conveniencia debe evaluarse con evidencia: se diseñó, implementó, evolucionó y midió una solución, y se contrastó con alternativas para decidir con criterio cuándo aplicar cada una.

Muchas gracias por la orientación y por la oportunidad de profundizar en estos temas.

Respetuosamente,

Alejandro De Mendoza